

ESTUDIO — Los flujos del sistema, contados como historias

Material de estudio para el final del TFC Bajoneá. **Este es el documento más importante para rendir.** Cada flujo está contado como una historia que podés relatar de memoria, mencionando qué controller recibe, qué service procesa, qué se valida, qué tablas se tocan y qué le vuelve al usuario.

Basado en el código real del backend y el frontend.

Cómo usar este documento

Cada flujo tiene la misma estructura:

1. **La historia**, contada en criollo.
2. **El recorrido técnico**, paso a paso.
3. **Las reglas de negocio clave** que se aplican.
4. **Qué le vuelve al usuario**.
5. **Lo que conviene destacar** si te preguntan.

Los flujos están ordenados por probabilidad de que te pregunten. Si tenés poco tiempo, estudiá los primeros cinco.

FLUJO 1 — Un cliente se registra

El más pedido en una mesa, porque toca casi todas las capas.

La historia

Ana entra a Bajoneá desde el celular y toca "Registrarme". Le aparece un formulario en pasos: primero sus datos personales, después su dirección, y al final su email y contraseña. Escribe todo, y mientras escribe la app le va marcando en rojo lo que está mal — el DNI con puntos, el teléfono corto. Cuando toca "Crear cuenta", la pantalla le dice que revise su correo. Ana abre el mail, ve un código de 6 dígitos, vuelve a la app, lo tipea, y recién ahí su cuenta queda activa. Ya puede loguearse.

El recorrido técnico

Antes de tocar el servidor

El navegador valida todo con `validators.js` : el DNI (7-8 dígitos), el email, la contraseña (8-72 con mayúscula, minúscula y número), el teléfono (`+549` + 10 dígitos), el CUIT si fuera comercio. **Nada de esto es la seguridad real** — es para que Ana no espere el viaje al servidor para enterarse de un error de tipeo.

Si Ana subió foto de perfil, ahí ya se ejecutó un mini-flujo: `POST /auth/registro/cliente/foto-firma` → sube la foto **directo a Cloudinary** → guarda la URL para mandarla en el registro.

El request

`POST /api/v1/auth/registro/cliente`

Con un `RegistroClienteRequestDTO` adentro.

Paso 1 — Los setters normalizan antes de validar

Antes de que Bean Validation mire nada, los setters manuales del DTO limpian:

- El nombre: espacios de sobra colapsados.
- El DNI: `"12.345.678"` → `"12345678"` .
- El teléfono: sin paréntesis ni guiones.
- El email: trim y minúsculas.

Esto importa: sin normalizar, alguien que escribe solo espacios en el nombre pasaría el `@NotBlank` y saltaría el error de formato, que confunde.

Paso 2 — `@Valid` dispara Bean Validation

Si algo falla → `MethodArgumentNotValidException` → el `GlobalExceptionHandler` devuelve **400** con un mapa `{campo: mensaje}` que el frontend pinta debajo de cada input.

Y acá hay un detalle: el handler **ordena** las violaciones para que los mensajes de "obligatorio" ganen siempre. Sin eso, el mismo campo vacío devolvía a veces "es obligatorio" y a veces "solo puede contener letras", de forma no determinística.

Paso 3 — `AuthController.registrarCliente` delega

El controller no hace nada más que llamar a `registroService.registrarCliente(request)` .

Paso 4 — RegistroService valida contra la base

Validación	Si falla
¿El email ya existe? (existsByEmail)	409 "Ya existe una cuenta registrada con ese email"
¿El DNI ya existe? (existsByDni)	409 "Ya existe una cuenta registrada con ese DNI"
¿La localidad existe? (findById)	404 "La localidad indicada no existe"

Paso 5 — Se crea la cadena completa de entidades

Todo dentro de una sola transacción (@Transactional).

1. Usuario

→ email, passwordHash (BCrypt), rol=CLIENTE, estado=PENDIENTE
2. Persona

→ @MapsId, mismo id que Usuario
3. PersonaFisica

→ nombre y apellido en Title Case, DNI, fecha, teléfono
4. Cliente

→ @MapsId, mismo id
5. Direccion

→ asociada al cliente, marcada como principal
6. Token

→ código de 6 dígitos, tipo VERIFICACION_EMAIL, vence en 24 hs

Los 4 primeros comparten el mismo id. Si Usuario tiene id 42, Persona , PersonaFisica y Cliente también.

Paso 6 — Se manda el email

EmailService.enviarVerificacion(email, token) → API de Resend.

Detalle importante: si el envío falla, **no se relanza la excepción**, solo se loguea. Si se relanzara, el @Transactional haría rollback y Ana perdería la cuenta recién creada porque el correo no salió. El token ya está persistido; puede pedir el reenvío.

Paso 7 — La respuesta

201 Created + UsuarioResponseDTO (sin el hash de la contraseña) + el mensaje "*Cliente registrado correctamente, verificá tu email*".

Paso 8 — La verificación

Ana escribe el código en el input de 6 dígitos (otp.js), que avanza solo entre casillas y acepta pegar el código completo.

POST /api/v1/auth/verificar { email, codigo }

AuthService.verificarEmailConCodigo :

1. Busca al usuario por email.
2. Busca su token pendiente más reciente de tipo VERIFICACION_EMAIL .
3. ¿Venció? → 409 "El código venció. Solicitá uno nuevo."
4. ¿No coincide? → suma un intento fallido, y devuelve cuántos quedan. **A los 5, el token se quema.**
5. ¿Coincide? → marca el token UTILIZADO y pone usuario.estado = ACTIVO .

Las reglas de negocio clave

1. La cuenta nace PENDIENTE , no ACTIVO . Sin verificar el email no podés loguearte.
2. Email y DNI únicos, validados en el service y garantizados por el UNIQUE de la base.
3. La contraseña se hashea con BCrypt, nunca se guarda en claro.
4. El código es de 6 dígitos (no un UUID) porque se tipea a mano.
5. El código tiene vencimiento y límite de intentos.

Lo que conviene destacar

La cadena de identidad. Es lo más llamativo del modelo: para dar de alta un cliente se crean 4 filas en 4 tablas, todas con el mismo id, gracias a @MapsId . Es herencia por tabla unida.

El email que no rompe la transacción. Es una decisión de diseño que muestra criterio: el envío de correo es un efecto secundario, no parte de la operación de negocio.

FLUJO 2 — El login y la autenticación con JWT

El segundo más preguntado, y el que más conceptos toca.

La historia

Ana escribe su email y contraseña. El sistema verifica que sea ella, le da un "carnet digital" firmado, y **anota en una tabla que abrió una sesión**. A partir de ahí, cada vez que Ana pide algo, muestra el carnet. El sistema verifica dos cosas: que el carnet sea auténtico, y que **esa sesión siga abierta**. Si Ana entra desde otro dispositivo, la sesión anterior se cae sola y en la primera pantalla le aparece un cartel explicándole por qué.

El recorrido — parte A: el login

```
POST /api/v1/auth/login { email, password }
```

AuthService.login(request, ip, userAgent)

Paso 1 — Busca al usuario CON BLOQUEO:

```
usuarioRepository.findByEmailConBloqueo(request.getEmail())
```

Es un `SELECT ... FOR UPDATE` : **bloquea esa fila** hasta terminar la transacción. Si el email no existe → `CredencialesInvalidasException` con **el mismo mensaje** que si la contraseña estuviera mal.

Paso 2 — Valida el estado de la cuenta:

Estado	Respuesta
ACTIVO	Pasa
PENDIENTE	409 "Verificá tu email antes de iniciar sesión"
BLOQUEADO	409 "Cuenta bloqueada. Recuperá tu contraseña para desbloquearla"
INACTIVO	409 "Cuenta inactiva. Solicitá la reactivación de tu cuenta"
SUSPENDIDO	409 "Cuenta suspendida"

Paso 3 — Compara la contraseña con `passwordEncoder.matches(...)` . **No se descripta nada**: se hashea lo que escribió Ana y se compara con el hash guardado.

Si falla:

- Suma 1 a `intentosFallidos` .
- Si llegó a 3: pasa a `BLOQUEADO` , cierra su sesión activa, y si es un **DUENO**, su comercio pasa a `CERRADO_TEMPORALMENTE` .
- Lanza la excepción **con los intentos restantes en el `data`** , para que la pantalla pueda mostrar "te quedan 2 intentos".

Si acierta:

- Resetea `intentosFallidos = 0` .
- Actualiza `fechaUltimoAcceso` .

Paso 4 — Cierra la sesión anterior si había una (`tipoCierre = FORZADO`).

Paso 5 — Crea la **sesion nueva** con `activa = true` , la IP y el navegador.

Paso 6 — Genera el JWT:

```
jwtService.generarToken(usuario, sesion.getId())
```

Con los claims: `sub` (email), `userId` , `rol` , **`sesionId`** , `iat` , `exp` (24 horas).

Paso 7 — Devuelve 200 + `LoginResponseDTO` con el token y los datos del usuario.

Del lado del navegador

```
auth.js guarda el token en localStorage y llama a resolverHomePorRol(usuario) :
```

- CLIENTE** → el catálogo.
- ADMINISTRADOR** → `admin-dashboard.html` .
- DUENO** → depende del **estado de su comercio**: aprobado va al dashboard, pendiente o rechazado a las pantallas correspondientes.

El recorrido — parte B: cada request posterior

```
GET /api/v1/carrito
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
```

1. RateLimitFotoRegistroFilter

→ no es ruta de firma, pasa de largo
2. JwtAuthenticationFilter

→ a. ¿Firma válida? no → 401

b. ¿Sesión activa? no → 401

c. arma `AuthenticatedUser` y lo pone en el contexto
3. Reglas de SecurityConfig

→ ¿el rol alcanza? no → 403
4. CarritoController.verCarrito()

El paso 2b es la particularidad del proyecto: el filtro consulta `Sesion.activa` **en la base**, en cada request. Un JWT clásico no hace eso.

Las reglas de negocio clave

1. Bloqueo a los 3 intentos fallidos.
2. Sesión única por cuenta: un login nuevo cierra el anterior.
3. Al bloquear la cuenta, se cierra la sesión (por si el atacante ya había entrado).
4. Si el bloqueado es un dueño, su comercio se cierra solo — si no puede entrar, no puede atender pedidos.
5. Nunca se dice si el email existe — mismo mensaje en los dos casos.

Lo que conviene destacar

"¿Por qué la tabla `Sesion` si el JWT es stateless?"

Esa es la pregunta de este flujo. La respuesta:

Un JWT puro no se puede revocar: una vez emitido vale hasta que expire, y si se lo roban a alguien sigue sirviendo 24 horas. Le agregué el claim `sessionId` y una tabla `Sesion`. En cada request el filtro chequea que esa sesión siga activa, así puedo invalidar un token de verdad poniendo `activa = false`. Lo uso en cuatro casos: logout, bloqueo de cuenta, cambio de contraseña y login concurrente.

El precio es una consulta a la base en cada request, que es justamente lo que un JWT stateless busca evitar. Lo acepté porque poder cerrar sesiones de verdad vale más que ahorrar esa consulta a esta escala.

El bug del `@Transactional` — la mejor anécdota del proyecto

El contador de intentos fallidos nunca subía. El método es `@Transactional`: incrementaba el contador y después lanzaba la excepción de credenciales inválidas. Spring, al ver una excepción unchecked, hacía rollback de toda la transacción... y con eso deshacía el incremento. La cuenta nunca se bloqueaba, podías probar contraseñas infinitas.

Se arregló con `@Transactional(noRollbackFor = CredencialesInvalidasException.class)`, acotado a esa clase puntual para no perder el rollback en el resto de los errores.

El detalle del bloqueo pesimista

Uso `SELECT ... FOR UPDATE` en el login. Sin eso, dos requests simultáneos con la contraseña mal leerían el contador en 2, ambos escribirían 3, y se perdería un intento. Con el bloqueo, el segundo espera y lee el valor ya actualizado.

FLUJO 3 — Un cliente hace un pedido, de principio a fin

El flujo de negocio más completo. Toca 6 tablas y coordina 4 services.

La historia

Ana entra al catálogo y ve los comercios abiertos primero. Toca "Pizzas del Sur", elige una muzzarella y toca "+". El producto va al carrito. Elige también una empanada. Después ve otro comercio y quiere agregar algo de ahí — el sistema le avisa que tiene que vaciar el carrito primero, porque un pedido es de un solo comercio.

Va al carrito, revisa, y toca "Continuar". En el checkout elige delivery, confirma su dirección, y toca "Confirmar pedido". La app le muestra el número de pedido. **Al mismo tiempo, en el celular del dueño de la pizzería, aparece una notificación: "Nuevo pedido recibido de Ana Pérez".**

El recorrido — parte A: armar el carrito

```
POST /api/v1/carrito/items { productoId, cantidad, nota }
```

`CarritoService.agregarItem(usuarioId, request)`

Paso 1 — Obtiene o crea el carrito. El carrito no se crea en el registro, se crea perezosamente la primera vez. Así no llenás la base de carritos vacíos.

Paso 2 — Busca el producto → 404 si no existe.

Paso 3 — ¿Está DISPONIBLE ? Si está AGOTADO O DESCONTINUADO → 409.

Paso 4 — ¿El comercio acepta pedidos? `comercioService.validarAceptaPedidos(comercio)` valida dos cosas:

- Que el comercio esté APROBADO .
- Que esté abierto en este momento, comparando la hora actual contra sus Horario .

Paso 5 — La regla del comercio único:

- Si el carrito está vacío (`comercio == null`) → se ata a este comercio.
- Si ya tiene otro → 409: *"El carrito ya tiene productos de otro comercio. Vacía el carrito para agregar de un comercio distinto."*

Paso 6 — ¿Ya está ese producto en el carrito?

- Sí → suma la cantidad, clampeada a 20. No rechaza.
- No → crea un `ItemCarrito` nuevo.

Devuelve el carrito completo actualizado, no solo el ítem — así el frontend repinta todo con una sola llamada.

El recorrido — parte B: confirmar el pedido

POST /api/v1/pedidos/cliente { tipoEntrega, direccionId }

PedidoService.confirmarPedido(usuarioId, request)

Fase 1 — Validar que se pueda:

Validación	Si falla
¿Existe el cliente?	404
¿Existe el carrito?	409 "El carrito está vacío"
¿Tiene ítems?	409 "El carrito está vacío"
¿El comercio acepta pedidos? (aprobado + abierto)	409
Si es DOMICILIO , ¿el comercio hace delivery?	409
Si es RETIRO , ¿el comercio permite retiro?	409
Si es DOMICILIO , ¿mandó direccionId ?	409
¿La dirección existe, es del cliente, y no está eliminada?	404

Fase 2 — Calcular:

- Valida que ningún ítem supere el tope de \$99.999.999.
- Suma el subtotal de todos los ítems.
- Los cargos de servicio quedan en **cero** (la lógica de comisiones pertenece al tramo de MercadoPago, todavía sin implementar).
- Valida que el total tampoco supere el tope.

Fase 3 — Persistir (todo en una transacción):

1. Pedido → estado=PENDIENTE, pagoEstado=PENDIENTE, subtotal, total, fecha
2. DetallePedido → uno por cada ítem, COPIANDO el precioUnitario
3. Vaciar el carrito
4. Notificacion → al dueño del comercio: "Nuevo pedido recibido de Ana Pérez."

Devuelve 201 Created + PedidoResponseDTO con el número de pedido.

El recorrido — parte C: el comercio responde

El dueño ve la notificación (el polling la trajo en menos de 15 segundos) y entra a sus pedidos.

Si acepta:

PUT /api/v1/pedidos/comercio/{id}/aceptar

PedidoService.aceptarPedido :

1. Verifica que el pedido **sea de su comercio** → si no, 404.
2. Verifica que esté PENDIENTE → si no, **409 "El pedido ya fue resuelto"** (esto evita el doble clic).
3. Pasa a **EN_PREPARACION** .
4. **Notifica al cliente:** "Tu pedido a Pizzas del Sur fue aceptado y está en preparación."

Si rechaza:

PUT /api/v1/pedidos/comercio/{id}/rechazar { motivo, comentario }

Igual, más una validación propia: si el motivo es OTRO , el comentario es obligatorio.

Pasa a RECHAZADO , guarda el motivo y el comentario, y notifica al cliente con la etiqueta legible del enum ("Alto volumen de pedidos", no ALTO_VOLUMEN_PEDIDOS).

Y del lado de Ana

No tiene que recargar nada: el polling de notificaciones.js levanta la notificación sola en menos de 15 segundos.

Las reglas de negocio clave

1. **Un carrito, un comercio.** Cada comercio prepara y entrega por su cuenta.
2. **El carrito se crea perezosamente.**
3. **No se puede pedir a un comercio cerrado o no aprobado.**
4. **La dirección tiene que ser tuya** — validación triple.

- 5. El precio se congela en el `DetallePedido` .
- 6. Un pedido solo se puede resolver una vez.

Lo que conviene destacar

El snapshot de precio

Cuando se confirma el pedido, el precio del producto se **copia** al `DetallePedido` , y ese campo tiene `updateable = false` . Si el comercio sube el precio mañana, el pedido de hoy sigue mostrando lo que se pagó. Un pedido es un documento histórico; si apuntara al precio actual del producto, el historial de compras cambiaría solo cada vez que el comercio ajusta precios.

Contrastalo con el carrito, que **no** guarda el precio: el carrito es algo vivo, querés ver el precio de hoy. El pedido es algo cerrado.

La validación de la dirección

No alcanza con que la dirección exista: valido que sea **del cliente que está pidiendo**. Sin ese chequeo, alguien podría mandar el `direccionId` de otro y hacer que le lleven el pedido a la casa de un desconocido. Y devuelvo 404, no 403, para no confirmar que esa dirección existe.

La deuda técnica del resumen del día, dicha de frente

El dashboard del comercio cuenta `EN_PREPARACION` como "facturado hoy". Es una deuda técnica documentada: como todavía no está implementado el estado `ENTREGADO` , no hay un estado final de éxito contra el cual medir. Lo que sí está correcto es que `RECHAZADO` nunca suma.

FLUJO 4 — Un dueño da de alta un producto

La historia

Marcelo, dueño de la pizzería, entra a "Mis productos" y toca "+". Escribe el nombre, la descripción, el precio, elige la categoría "Pizzas" de un selector y le pone los tags "abundante" y "vegetariano". Guarda. Después sube las fotos: elige una del celular, la recorta en un editor 4:3, y la sube. Repite hasta 5. Al intentar la sexta, el sistema le avisa que llegó al máximo.

El recorrido — parte A: crear el producto

`POST /api/v1/productos`

El selector de categoría es una historia en sí misma: para llenarlo, el frontend llama a `GET /categorias` . Ese endpoint era **ADMINISTRADOR-only**, así que Marcelo no podía verlo. Se resolvió en `SecurityConfig` poniendo una regla más específica **antes** que la genérica:

```
.requestMatchers(HttpMethod.GET, "/api/v1/categorias/**", "/api/v1/tags/**").authenticated()
.requestMatchers("/api/v1/categorias/**", "/api/v1/tags/**", ...).hasRole("ADMINISTRADOR")
```

El GET lo puede hacer cualquier autenticado; crear, editar y borrar sigue siendo del admin.

ProductoService.crearProducto(usuarioId, request)

- Resuelve el comercio del usuario (`comercioRepository.findByDuenoId(usuarioId)`).
- Busca la categoría → 404 si no existe.
- Crea el producto: nombre en Title Case, **estado** `DISPONIBLE` (lo decide el service, no viene en el DTO).
- Asigna los tags creando una fila en `ProductoTag` por cada uno.

Devuelve 201 + `ProductoResponseDTO` con el id, que el frontend necesita para el paso siguiente.

El recorrido — parte B: las fotos, en dos pasos

Por qué las fotos van después: para armar la carpeta de Cloudinary hace falta el `productoId` , que recién existe cuando el producto está guardado.

Paso 1 — Pedir la firma

`POST /api/v1/productos/{id}/cloudinary/firma`

`CloudinaryService.generarFirmaImagenProducto(comercioId, productoId)` :

- Cuenta cuántas imágenes tiene → si ya hay 5, **409** antes de firmar (para no dejar que Cloudinary reciba un archivo que se va a descartar).
- Firma para la carpeta `productos/{comercioId}/{productoId}/` , con el `apiSecret` que **nunca sale del servidor**.

Paso 2 — El frontend sube DIRECTO a Cloudinary

```
await fetch(`https://api.cloudinary.com/v1_1/${firma.cloudName}/image/upload`, {...});
```


Fijate: el `fetch` apunta a `api.cloudinary.com` , no al backend. El servidor nunca ve el archivo.

Cloudinary aplica el **Upload Preset** `bajonea_imagenes_mvp` : solo `jpg/png/webp`, máximo 5 MB, y reduce el ancho a 1200px con `quality: auto` .

Paso 3 — Registrar la URL

```
POST /api/v1/productos/{id}/imagenes { url, orden, esPrincipal }
```

`ProductoService.agregarImagen` :

1. Verifica que el producto sea de su comercio → 404 si no.
2. Segunda validación del límite de 5 (por si dos firmas se pidieron casi en simultáneo).
3. Si es la primera imagen, la marca como principal automáticamente.
4. Guarda la fila en `imagen_producto` .

Las reglas de negocio clave

1. Un producto nace `DISPONIBLE` .
2. Máximo 5 imágenes, validado dos veces.
3. La primera imagen por orden es siempre la principal, recalculado tras cada operación.
4. Al editar, hace falta al menos una foto. Una regla de UX convertida en regla de negocio.
5. No se puede editar un producto `DESCONTINUADO` .
6. Solo podés tocar productos de tu comercio → 404 si no.

La máquina de estados del producto

```
DISPONIBLE → { AGOTADO, DESCONTINUADO }
AGOTADO → { DISPONIBLE, DESCONTINUADO }
DESCONTINUADO → { } ← terminal, sin salida
```

Está declarada como un `Map` , no como una cadena de `if` . Agregar un estado es agregar una línea.

El efecto dominó de agotar un producto

Cuando Marcelo marca una pizza como `AGOTADA` , el service ejecuta `limpiarCarritosActivos` :

1. Busca **todos los ítems de todos los carritos** que tengan ese producto.
2. **Notifica a cada cliente afectado:** *"El producto 'Muzzarella' ya no está disponible y fue eliminado de tu carrito."*
3. Después borra los ítems.
4. Si algún carrito quedó vacío, **le resetea el comercio a null**.

El orden importa: notificar antes de borrar, porque después ya no sabrías a quién avisar.

El paso 4 es un bug que se encontró y corrigió: sin él, un cliente quedaba con el carrito vacío pero "atado" a un comercio, sin poder comprar en otro.

Lo que conviene destacar

La subida de imágenes es en dos pasos y **el archivo nunca pasa por mi servidor**: el frontend pide una firma, sube directo a Cloudinary, y me manda solo la URL. Eso ahorra ancho de banda y disco. La firma está atada a una carpeta derivada del `comercioId` y el `productoId` que salen del JWT, así que un comercio no puede subir a la carpeta de otro.

FLUJO 5 — Un administrador aprueba o rechaza un comercio

La historia

Marcelo registró su pizzería, pero todavía no aparece en el catálogo — está pendiente. El administrador entra a su panel, ve "3 comercios pendientes", abre el de Marcelo y revisa: la razón social, el CUIT, la condición de IVA, quién es el representante legal, la dirección, los horarios y las redes sociales. Todo está en orden, así que aprueba. **En el celular de Marcelo aparece la notificación: "Tu comercio fue aprobado."** Al entrar de nuevo, ya no ve la pantalla de "pendiente" sino su dashboard, y su pizzería ya está en el catálogo público.

El recorrido

Antes: por qué el comercio nace pendiente

En `RegistroService.registrarComercio` , el comercio se crea con `estado = EstadoComercio.PENDIENTE` . Y `CatalogoService` solo lista los `APROBADO` . Por eso no aparece en la vidriera.

El panel del admin

```
GET /api/v1/administrador/metricas → los 5 números del dashboard
GET /api/v1/administrador/comercios/pendientes → la cola de aprobación
```


Detalle a notar: el administrador **no filtra por "sus" comercios**. A diferencia del cliente (que ve solo sus pedidos) o del dueño (que ve solo sus productos), el admin ve **todo el sistema**. Es una consulta global por diseño.

La resolución

```
PUT /api/v1/administrador/comercios/{comercioId}/resolver { aprobar, motivo }
```

AdministradorService.resolverAprobacion(...)

- Busca el comercio → 404 si no existe.
- ¿Está `PENDIENTE` ? Si no → 409 "El comercio ya fue resuelto" (*evita el doble clic*).
- Si rechaza, ¿mandó motivo? Si no → 400 "El motivo es obligatorio al rechazar un comercio".
- Busca al administrador que está resolviendo.
- Cambia el estado a `APROBADO` o `RECHAZADO` .
- Crea una fila en `HistorialEstadoComercio` : el comercio, **quién** resolvió, el estado de origen, el de destino, el motivo y la fecha.
- Notifica al dueño.

Devuelve 200, sin data.

Del lado de Marcelo

Al entrar, `initComercioEstadoPagina` consulta su perfil y lo manda al dashboard (antes lo mandaba a "pendiente"). Y su comercio ya aparece en `GET /catalogo/comercios` .

Si lo hubieran rechazado, `ComercioService.obtenerMotivoRechazo` busca la última transición del historial y le muestra el motivo.

Las reglas de negocio clave

- Solo se puede resolver un comercio `PENDIENTE` .
- El motivo es obligatorio al rechazar — validación condicional, imposible con Bean Validation.
- Cada resolución deja un registro de auditoría inmutable.
- Solo los `APROBADO` aparecen en el catálogo.
- El dueño se entera por notificación.

Lo que conviene destacar

El historial es la única fuente del motivo

El modelo de datos eliminó la columna `Comercio.motivo_rechazo` a favor de la tabla `HistorialEstadoComercio` . Es una tabla **append-only**: solo se insertan filas, nunca se modifican. Queda registrado quién aprobó o rechazó, cuándo y por qué. Para mostrarle el motivo al comercio, busco su última transición a `RECHAZADO` .

La cadena de identidad, otra vez

Para notificar al dueño hay que recorrer cuatro relaciones:

```
comercio.getDueno().getPersonaJuridica().getPersona().getUsuario().getId()
```

Es el precio de la normalización. Un buen ejemplo concreto para mostrar que entendés el modelo.

FLUJO 6 — Recuperar una contraseña olvidada

La historia

Ana no se acuerda de su contraseña. Toca "Olvidé mi contraseña", escribe su email, y la app le dice "*Si existe una cuenta asociada a ese email, vas a recibir un código*". Le llega un mail con un código de 6 dígitos, que dura 30 minutos. Lo escribe y la app le confirma que es válido — **todavía sin pedirle la contraseña nueva**. Recién en la pantalla siguiente escribe la nueva. Al confirmar, **su sesión anterior se cierra** y tiene que loguearse de nuevo.

El recorrido — tres pasos, no dos

Paso 1 — Pedir el código

```
POST /api/v1/auth/recuperar-password { email }
```

```
usuarioRepository.findByEmail(request.getEmail()).ifPresent(usuario -> {
    Token token = generarToken(usuario, RECUPERACION_PASSWORD, ahora + 30 min);
    emailService.enviarRecuperacionPassword(usuario.getEmail(), token.getToken());
});
```

Si el email no existe, no pasa absolutamente nada. El controller devuelve el mismo 200 con el mismo mensaje ambiguo.

Paso 2 — Validar el código, SIN cambiar nada

POST /api/v1/auth/recuperar-password/validar-codigo { email, codigo }

Por qué existe este paso intermedio: es puramente de experiencia de usuario. Te avisa que el código está mal antes de que escribas la contraseña nueva dos veces. Sin él, tendrías que completar todo el formulario para recién ahí enterarte de que el código era incorrecto.

Valida pero no consume el token — solo el paso 3 lo quema.

Paso 3 — Cambiar la contraseña

POST /api/v1/auth/recuperar-password/confirmar { email, codigo, nuevaPassword }

AuthService.confirmarRecuperacionPassword :

1. Valida el código de nuevo.
2. Consume el token (UTILIZADO + fechaUso).
3. Guarda la contraseña nueva hasheada.
4. Pone el estado en ACTIVO ← esto es lo que desbloquea una cuenta bloqueada.
5. Resetea intentosFallidos = 0 .
6. Cierra la sesión activa (FORZADO).
7. Si es un dueño, restaura su comercio de CERRADO_TEMPORALMENTE a APROBADO .

Las reglas de negocio clave

1. El código dura 30 minutos (el más corto de los tres tipos).
2. Máximo 5 intentos, después el token se quema.
3. Es de un solo uso.
4. Cambiar la contraseña cierra la sesión — si te la robaron, el atacante queda afuera.
5. Este flujo desbloquea una cuenta bloqueada.
6. Nunca se revela si el email existe.

Lo que conviene destacar

La enumeración de usuarios — y el bug que la causaba

Los endpoints de recuperación, reactivación y reenvío devuelven siempre el mismo mensaje, exista o no el email. Si respondieran distinto, alguien podría probar mails y armar una lista de quién está registrado.

Y había un bug real ahí: antes lanzaban RecursoNoEncontradoException , que llegaba al handler y devolvía un 404 real — contradiciendo el mensaje ambiguo del controller. Se corrigió usando ifPresent : si el email no existe, no pasa nada y el controller devuelve el mismo 200.

El caso más fino, en la reactivación

Hubo un segundo caso, más sutil. solicitarReactivacionCuenta solo generaba token si la cuenta estaba INACTIVO . Entonces una cuenta en otro estado no tenía token contra el cual contar intentos, y el paso de confirmación fallaba distinto (sin intentosRestantes , sin bloqueo). Esa diferencia de comportamiento filtraba el estado real.

La solución fue generar el token siempre, y que la confirmación decida si corresponde cambiar el estado: si la cuenta no estaba inactiva, el código se consume igual pero sin efecto real. Desde afuera todos los casos se ven idénticos.

FLUJO 7 — Las notificaciones in-app

La historia

Marcelo tiene abierto su dashboard. Ana confirma un pedido. Sin que Marcelo toque nada, a los pocos segundos la campanita del header se pone en "1". Toca la campanita, ve "Nuevo pedido recibido de Ana Pérez", hace clic y va directo al pedido.

El recorrido

El backend crea la notificación

Todo pasa por NotificacionService.crear(...) . Antes estaba copiado y pegado en tres services; se centralizó para que un cambio de estructura (como cuando se agregó el par entidadTipo / entidadId) se haga en un solo lugar.

Los sitios reales de creación en el sistema:

Dónde	Cuándo	Tipo
PedidoService.confirmarPedido	El cliente confirma	NUEVO_PEDIDO (al comercio)
PedidoService.aceptarPedido	El comercio acepta	PEDIDO_ACEPTADO (al cliente)
PedidoService.rechazarPedido	El comercio rechaza	PEDIDO_RECHAZADO (al cliente)
ProductoService.limpiarCarritosActivos	Un producto se agota	PRODUCTO_REMOVIDO_CARRITO
AdministradorService.resolverAprobacion	Se resuelve un comercio	COMERCIO_APROBADO / COMERCIO_RECHAZADO

El polling

`notificaciones.js` tiene un `setInterval` que **cada 15 segundos** llama a:

```
GET /api/v1/notificaciones/no-leidas/contador
```

Y actualiza el numerito. **Devuelve solo un `long`**, con un `COUNT(*)` en SQL — es el endpoint más llamado del sistema, así que se hizo lo más liviano posible.

La lista completa (`GET /notificaciones`) solo se trae cuando el usuario abre la campanita.

La referencia polimórfica

Cada notificación lleva `entidadTipo` + `entidadId` :

```
entidadTipo = PEDIDO,  entidadId = 42  → "es sobre el pedido 42"
entidadTipo = COMERCIO, entidadId = 7   → "es sobre el comercio 7"
```

Es lo que le permite al frontend saber a qué pantalla llevarte al hacer clic. Antes había un campo `pedido_id` fijo, que solo servía para notificaciones de pedido.

Lo que conviene destacar

Polling vs WebSockets

No es tiempo real de verdad, uso polling: el frontend pregunta cada 15 segundos. La alternativa serían WebSockets, donde el servidor te avisa cuando pasa algo — instantáneo, pero mucho más complejo: una conexión abierta por usuario y manejo de reconexiones.

Para esta escala, polling alcanza: 15 segundos de demora en enterarte de un pedido no rompe nada. Lo que sí optimicé es que el endpoint del polling devuelva solo un número, no la lista.

El trade-off de la referencia polimórfica

La ventaja es que una sola tabla sirve para notificaciones de cualquier entidad. La desventaja es que **la base no puede poner una foreign key** — no sabe a qué tabla apunta `entidadId` . Si se borrara el pedido 42, la notificación quedaría apuntando a la nada y MySQL no avisaría. La integridad queda a cargo de la aplicación. Es el trade-off clásico de este patrón.

FLUJO 8 — Un comercio se registra

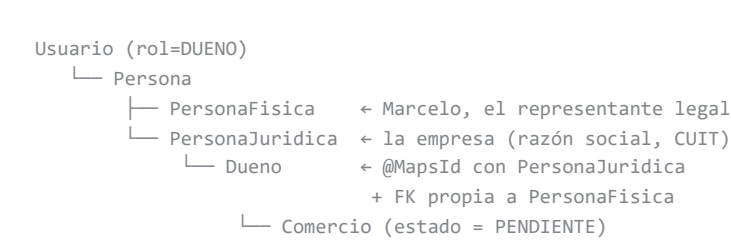
Variante del Flujo 1, pero mucho más compleja. Vale la pena tenerla aparte.

La historia

Marcelo quiere sumar su pizzería. El wizard le pide, en pasos: los datos legales de la empresa (razón social, CUIT, condición de IVA, tipo de sociedad, domicilio fiscal, fecha de inicio de actividades), los datos del comercio (nombre de fantasía, descripción, teléfono, email de contacto, tipo, si hace delivery o retiro), **su horario de atención día por día**, sus redes sociales, sus datos personales como representante legal, y una foto de la pizzería. Al final, sus credenciales de acceso. Cuando termina, la app le dice que verifique su email y que **su comercio queda pendiente de aprobación**.

Lo que hace distinto a este flujo

1. Se crean DOS personas colgando de la misma **Persona**



El **Dueno** tiene dos relaciones a la vez: `@MapsId` con la persona jurídica ("*yo soy esta empresa*") y una `@OneToOne` con FK propia hacia la persona física ("*esta persona me representa*"), marcada `unique` para que una persona no represente a dos dueños.

2. Hay DOS emails distintos, a propósito

Campo	Para qué
email	La credencial de login de Marcelo.
emailContacto	El email público de la pizzería, que ve el cliente.

3. Tres validaciones de unicidad, no dos

Email, CUIT y DNI del representante.

4. Los horarios tienen una validación no trivial

RegistroService.validarHorarios :

Regla A: la hora de cierre tiene que ser posterior a la de apertura.

Regla B — los horarios del mismo día no se pueden superponer:

```
boolean seSuperponen = actual.getHoraApertura().isBefore(otro.getHoraCierre())
                        && otro.getHoraApertura().isBefore(actual.getHoraCierre());
```

Es el algoritmo clásico de solapamiento de intervalos: dos rangos se solapan si y solo si $a_1 < b_2$ && $b_1 < a_2$. Con dos comparaciones cubre todos los casos (uno adentro del otro, cruzados, parcialmente superpuestos).

Y el mensaje de error es excelente: *"Ya tenés un horario cargado el Miércoles de 12:00 a 15:00, que se superpone con este"* — le dice exactamente cuál es el conflicto.

5. Las redes sociales tienen sus propias reglas

- Al menos una, máximo 5.
- Sin tipos repetidos (no dos Instagram):

```
long tiposUnicos = redesSociales.stream().map(RedSocialRequestDTO::getTipo).distinct().count();
if (tiposUnicos != redesSociales.size()) throw new ValidacionException(...);
```

6. La foto de perfil es obligatoria

Y se sube **antes** de crear la cuenta, con la firma pública `POST /auth/registro/comercio/foto-firma` a la carpeta fija `comercios/pre-registro/` — porque el comercio todavía no tiene id.

Ese endpoint es público, así que está protegido por el `RateLimitFotoRegistroFilter` : máximo **5 firmas por minuto por IP**. Sin ese freno, cualquiera podría llenar (y facturar) la cuenta de Cloudinary con un script.

7. Se valida que ofrezca al menos una modalidad de entrega

ComercioValidaciones.validarModalidadesEntrega(aceptaDelivery, aceptaRetiro) — si las dos son `false` , error. **Un comercio al que nadie le puede pedir nada no tiene sentido.**

Esa validación está en una clase utilitaria porque la usan **dos** services: el de registro y el de edición de perfil.

8. El representante Sí tiene que ser mayor de edad

`@MayorDeEdad` en `fechaNacimientoRepresentante` , mientras que el cliente usa `@ValidarFechaNacimientoPlausible` (sin piso de edad).

Es una decisión de negocio explícita, no un descuido: el representante legal de una sociedad debe ser mayor por requisito legal; un cliente que pide comida no.

FLUJO 9 — Un comercio edita su perfil

La historia corta

Marcelo cambia el teléfono de la pizzería y desactiva el delivery porque se le rompió la moto.

El recorrido

PUT /api/v1/comercios/perfil

ComercioService.editarPerfil :

- Valida las modalidades (ComercioValidaciones) → si desactiva las dos, error.
- Actualiza nombre (Title Case), descripción, teléfono, email de contacto y las dos modalidades.

Lo que conviene destacar: lo que NO se puede editar

El DTO **no incluye** `razonSocial`, `cuit` ni `condicionIva`. Este endpoint es para el perfil **público**, no para los datos legales — cambiar un CUIT debería tener un proceso con verificación, no un formulario libre.

Y **tampoco incluye** `fotoPerfilUr1`, por un motivo sutil que vale la pena contar:

`@ValidarUrlCloudinary` valida el **dominio**, no la **propiedad**. Un comercio podría pegar la URL de una foto subida por otro y pasaría la validación. Por eso la foto tiene su propio par de endpoints con firma, donde la firma está atada al `comercioId` del token y garantiza que la imagen es suya. **La anotación cubre el dominio; la propiedad la garantiza el flujo de firma.**

FLUJO 10 — El administrador gestiona categorías y tags

La historia corta

El admin crea la categoría "Sushi". Después ve que "Comida japonesa" no la usa nadie y la da de baja. Más tarde se arrepiente y la reactiva.

El recorrido

Acción	Endpoint	Qué hace
Crear	<code>POST /categorias</code>	Valida nombre único, nace <code>activo = true</code> .
Editar	<code>PUT /categorias/{id}</code>	Valida unicidad solo si el nombre cambió .
Listar	<code>GET /categorias</code>	Devuelve todas, con su conteo de productos.
Dar de baja	<code>DELETE /categorias/{id}</code>	<code>activo = false</code> + <code>fechaBaja</code> .
Reactivar	<code>PUT /categorias/{id}/reactivar</code>	<code>activo = true</code> + <code>fechaBaja = null</code> .

Lo que conviene destacar

La baja lógica

El `DELETE` **no borra la fila**, pone `activo = false`. Si borrara de verdad, los productos que usaban esa categoría quedarían apuntando a algo que no existe. Con baja lógica desaparece de los listados nuevos pero los datos históricos quedan coherentes, y se puede reactivar. Se llama **soft delete**.

El bug clásico que se evitó en el `editar`

```
if (!categoria.getNombre().equals(request.getNombre())
    && categoriaRepository.existsByNombre(request.getNombre())) { ... }
```

Sin el primer chequeo, guardar la categoría **sin cambiarle el nombre** daría error — porque `existsByNombre` encontraría la propia categoría que estás editando. Es un error muy común en validaciones de unicidad en edición.

El conteo de productos

`cantidadProductos` es un dato **calculado** (`countByCategoriaId`), no una columna. Sirve para que el admin vea *"Pizzas — 12 productos asociados"*.

El sistema **no bloquea dar de baja una categoría en uso**. Es deliberado: como la baja es lógica y reversible, los productos siguen apuntando a algo que existe. Se le muestra el conteo al admin **para que decida informado**, no para prohibirle la acción.

FLUJO 11 — Navegar el catálogo público (sin estar logueado)

La historia

Un vecino de Río Grande, sin cuenta, entra a Bajoneá para ver qué hay. Ve la lista de comercios, con los abiertos primero. Puede filtrar, entrar a un comercio y ver sus productos. Recién cuando toca "agregar al carrito" le pide que se registre.

El recorrido

Los 4 endpoints son públicos (`SecurityConfig`, `RUTAS_PUBLICAS`):

Endpoint	Qué devuelve
GET /catalogo/comercios	Los comercios APROBADO .
GET /catalogo/comercios/{id}/productos	Sus productos, con filtros opcionales.
GET /catalogo/productos	Búsqueda global, paginada.
GET /catalogo/filtros	Las categorías y tags disponibles.

Lo que conviene destacar

El DTO público es distinto — y ese es el punto

`CatalogoService` usa `ComercioPublicoResponseDTO`, **nunca** `ComercioResponseDTO`. La diferencia es que el público **no lleva los datos del representante legal, incluido su DNI**. Ese dato no puede viajar sin autenticación.

Están en clases separadas justamente para que sea **imposible** que se cuele por error: para incluir el representante en el DTO público habría que agregarlo explícitamente.

El criterio de visibilidad de productos

```
findByComercioIdAndEstadoNot(comercioId, EstadoProducto.DESCONTINUADO)
```

Excluye `DESCONTINUADO` pero **no** `AGOTADO`. Un producto agotado **se muestra igual**, marcado por su estado, para que el frontend lo **deshabilite en vez de ocultarlo**. Si desapareciera, el cliente no sabría que existe; en gris con "agotado", sabe que el comercio lo tiene y que puede volver.

La decisión discutible, dicha de frente

`listarCatalogoGlobal` **filtra, mezcla y pagina todo en memoria, no en SQL**.

Es una decisión consciente y documentada: el catálogo del proyecto es chico, se trae completo y se procesa en Java, que es más simple que armar una query dinámica con `ORDER BY RAND()`.

Pero no escala. Con 10.000 productos, traerlos todos en cada request sería carísimo. La solución a esa escala sería paginación en SQL con `Pageable` y los filtros en el `WHERE`.

(Decirlo así, reconociendo el límite, es mucho mejor que defenderlo como si fuera óptimo.)

FLUJO 12 — Qué pasa cuando un comercio cierra (por horario o por bloqueo)

Un flujo transversal poco obvio, muy bueno para mostrar que entendés el sistema completo.

Los tres caminos por los que un comercio deja de recibir pedidos

Camino A — Está cerrado por horario

`ComercioService.estaAbiertoAhora(horarios)`:

```
DiaSemana diaHoy = DiaSemana.values()[ahora.getDayOfWeek().getValue() - 1];
return horarios.stream()
    .filter(h -> h.getDiaSemana() == diaHoy)
    .anyMatch(h -> !horaActual.isBefore(h.getHoraApertura())
        && horaActual.isBefore(h.getHoraCierre()));
```

Detalles:

- El `-1` alinea `DayOfWeek` de Java (que arranca en 1) con el índice del enum (que arranca en 0).
- El `anyMatch` es lo que hace funcionar el **horario partido**: si abre 12-15 y 20-00, basta con caer en cualquiera de los dos rangos.
- Sin horarios cargados → cerrado**. Mejor no aceptar un pedido que no se va a poder cumplir.

Se valida en **dos** momentos: al agregar al carrito y al confirmar el pedido.

Camino B — El dueño se bloqueó la cuenta

Si el dueño falla 3 veces la contraseña:

```
usuario.setEstado(EstadoUsuario.BLOQUEADO);
cerrarSesionActivaSiExiste(usuario, FORZADO);
propagarBloqueoAComercio(usuario);           // ← comercio APROBADO → CERRADO_TEMPORALMENTE
```

La lógica: si el dueño no puede entrar a la app, no puede aceptar pedidos. Dejarlo abierto en el catálogo generaría pedidos que nadie va a atender. **El sistema lo cierra solo**.

Y la operación inversa: cuando recupera la contraseña o reactiva la cuenta, `restaurarComercioSiCorresponde` lo vuelve a `APROBADO`. Recibe **de qué estado tiene que venir**, para no reactivar por error un comercio cerrado por otro motivo.

Camino C — El administrador lo rechazó o suspendió

El estado deja de ser `APROBADO` , así que `CatalogoService` no lo lista y `validarAceptaPedidos` lo rechaza.

Lo que conviene destacar

Hay tres caminos independientes por los que un comercio deja de recibir pedidos: horario, bloqueo de la cuenta del dueño, o decisión del administrador. Los tres convergen en el mismo método — `validarAceptaPedidos` — que se llama tanto al agregar al carrito como al confirmar el pedido. **La regla vive en un solo lugar**, en vez de estar duplicada en `CarritoService` y `PedidoService` .

Tabla resumen de los 12 flujos

#	Flujo	Controller	Service principal	Tablas que toca
1	Registro de cliente	<code>AuthController</code>	<code>RegistroService</code>	<code>usuario</code> , <code>persona</code> , <code>persona_fisica</code> , <code>cliente</code> , <code>direccion</code> , <code>token</code>
2	Login y JWT	<code>AuthController</code>	<code>AuthService</code>	<code>usuario</code> , <code>sesion</code>
3	Pedido completo	<code>CarritoController</code> + <code>PedidoController</code>	<code>CarritoService</code> + <code>PedidoService</code>	<code>carrito</code> , <code>item_carrito</code> , <code>pedido</code> , <code>detalle_pedido</code> , <code>notificacion</code>
4	Alta de producto	<code>ProductoController</code>	<code>ProductoService</code> + <code>CloudinaryService</code>	<code>producto</code> , <code>producto_tag</code> , <code>imagen_producto</code>
5	Aprobación de comercio	<code>AdministradorController</code>	<code>AdministradorService</code>	<code>comercio</code> , <code>historial_estado_comercio</code> , <code>notificacion</code>
6	Recuperar contraseña	<code>AuthController</code>	<code>AuthService</code>	<code>usuario</code> , <code>token</code> , <code>sesion</code> , <code>comercio</code>
7	Notificaciones	<code>NotificacionController</code>	<code>NotificacionService</code>	<code>notificacion</code>
8	Registro de comercio	<code>AuthController</code>	<code>RegistroService</code>	<code>usuario</code> , <code>persona</code> , <code>persona_fisica</code> , <code>persona_juridica</code> , <code>dueno</code> , <code>comercio</code> , <code>direccion</code> , <code>horario</code> , <code>red_social</code> , <code>token</code>
9	Editar perfil de comercio	<code>ComercioController</code>	<code>ComercioService</code>	<code>comercio</code>
10	Categorías y tags	<code>CategoriaController</code> / <code>TagController</code>	<code>CategoriaService</code> / <code>TagService</code>	<code>categoria</code> , <code>tag</code>
11	Catálogo público	<code>CatalogoController</code>	<code>CatalogoService</code>	<code>comercio</code> , <code>producto</code> , <code>imagen_producto</code> , <code>producto_tag</code>
12	Cierre de comercio	<i>(transversal)</i>	<code>ComercioService</code> + <code>AuthService</code>	<code>comercio</code> , <code>usuario</code> , <code>sesion</code>

Los 5 flujos que hay que saber sí o sí

Si tenés poco tiempo, priorizá en este orden:

1. **Login y JWT** (Flujo 2) — porque toca seguridad, que es lo más preguntado.
2. **Pedido completo** (Flujo 3) — porque es el flujo de negocio central.
3. **Registro de cliente** (Flujo 1) — porque muestra la cadena de identidad.
4. **Aprobación de comercio** (Flujo 5) — porque muestra el rol administrador y la auditoría.
5. **Alta de producto** (Flujo 4) — porque muestra el flujo de imágenes con Cloudinary.

Índice de flujos cubiertos en este documento

#	Flujo	Qué demuestra que entendés
1	Un cliente se registra	La cadena de identidad <code>@MapsId</code> y el ciclo de verificación.
2	Login y autenticación con JWT	JWT, sesión revocable, bloqueo, concurrencia.
3	Un cliente hace un pedido	El flujo de negocio central, snapshot de precio, notificaciones.
4	Un dueño da de alta un producto	Máquina de estados, Cloudinary, efecto en carritos.
5	Un admin aprueba/rechaza un comercio	Auditoría inmutable y validación condicional.
6	Recuperar contraseña	Tokens de un solo uso y prevención de enumeración de usuarios.
7	Las notificaciones in-app	Polling vs WebSockets, referencia polimórfica.
8	Un comercio se registra	Dos personas de una misma <code>Persona</code> , solapamiento de horarios.
9	Un comercio edita su perfil	Qué NO se puede editar, y por qué.
10	Categorías y tags	Baja lógica y el bug de unicidad en edición.
11	Catálogo público	DTOs distintos por audiencia, criterio de visibilidad.
12	Cuando un comercio cierra	Los tres caminos que convergen en una sola regla.